Sesión 7

GRADIENT BOOSTING

XGBoost, LightGBM y el estado del arte en tablas

Por

Carlos César Sánchez Coronel

2026

Índice general

Índice general **2**

1. Gradient Boosting (XGBoost, LightGBM) **3**
1.1. Logro de la sesión . . . 3
1.2. Introducción: de bagging a boosting . . . 3
1.3. El concepto de Boosting: intuición y AdaBoost . . . 3
1.4. Gradient Boosting: fundamento matemático . . . 4
1.4.1. Marco general . . . 4
1.4.2. Ajuste de los residuos (vista de regresión) . . . 4
1.4.3. Funciones de pérdida arbitrarias . . . 4
1.4.4. Algoritmo genérico de Gradient Boosting . . . 5
1.4.5. Componentes clave . . . 5
1.5. Implementaciones modernas: XGBoost, LightGBM y CatBoost . . . 5
1.5.1. XGBoost (eXtreme Gradient Boosting) . . . 5
1.5.2. LightGBM (Light Gradient Boosting Machine) . . . 6
1.5.3. CatBoost (Categorical Boosting) . . . 7
1.6. Comparación con Random Forest . . . 7
1.7. Hiperparámetros clave en XGBoost y LightGBM . . . 7
1.7.1. Hiperparámetros comunes . . . 7
1.7.2. Hiperparámetros específicos de XGBoost . . . 8
1.7.3. Hiperparámetros específicos de LightGBM . . . 8
1.8. Métricas de evaluación en boosting . . . 8
1.8.1. Clasificación . . . 8
1.8.2. Regresión . . . 8
1.8.3. Ejemplo comparativo de métricas . . . 9
1.9. Aplicaciones prácticas . . . 9
1.9.1. Predicción de clics en publicidad online (CTR) . . . 9
1.9.2. Competencias Kaggle . . . 9
1.9.3. Modelado de riesgo crediticio . . . 9
1.10. Caso integrado: Predicción de abandono de clientes (churn) . . . 10
1.10.1. Problema . . . 10
1.10.2. Datos . . . 10
1.10.3. Preprocesamiento . . . 10
1.10.4. Modelado y ajuste de hiperparámetros . . . 10
1.10.5. Resultados . . . 10
1.10.6. Importancia de características . . . 11
1.10.7. Conclusión del caso . . . 11
1.11. Implementación en Python . . . 11

1.11.1. XGBoost 11
1.11.2. LightGBM 12
1.12. Conexión con el resto del curso 13
1.13. Resumen 13

Capítulo 1

Gradient Boosting (XGBoost, LightGBM)

En esta sesión se aborda la familia de algoritmos de boosting, con énfasis en Gradient Boosting y sus implementaciones modernas y altamente eficientes: XGBoost, LightGBM y (brevemente) CatBoost. Se explican los fundamentos matemáticos, las innovaciones computacionales que los hacen tan potentes, los hiperparámetros clave, las métricas adecuadas y las aplicaciones prácticas. El objetivo es que el estudiante comprenda por qué estos modelos dominan las competiciones de machine learning y gran parte de la industria para datos tabulares.

1.1. Logro de la sesión

Dominar los algoritmos de boosting modernos y ajustar sus hiperparámetros para obtener modelos de alto rendimiento, comprendiendo las diferencias entre implementaciones y su aplicabilidad en diversos contextos.

1.2. Introducción: de bagging a boosting

En la sesión anterior se estudió Random Forest, un método de ensamble basado en bagging: los árboles se construyen en paralelo y sus predicciones se promedian. El boosting adopta una filosofía diferente: los modelos se construyen de forma secuencial, donde cada nuevo modelo intenta corregir los errores del conjunto previo. Esta corrección iterativa permite que el boosting alcance, en muchos problemas, un rendimiento superior al de bagging, aunque con mayor riesgo de sobreajuste si no se controla adecuadamente.

1.3. El concepto de Boosting: intuición y AdaBoost

El primer algoritmo de boosting exitoso fue AdaBoost (Adaptive Boosting). Su idea es asignar pesos a las observaciones: inicialmente todos iguales. En cada iteración:

1. Se entrena un clasificador débil (por ejemplo, un árbol de profundidad 1, llamado "stump").

2. Se aumenta el peso de las observaciones mal clasificadas.

3. Se repite.

La predicción final es una votación ponderada de todos los clasificadores. AdaBoost demostró que incluso clasificadores muy simples podían combinarse para obtener un modelo muy preciso.

Sin embargo, AdaBoost tiene limitaciones: es sensible a datos ruidosos y su formulación no se generaliza fácilmente a funciones de pérdida arbitrarias. El gradient boosting surge como una generalización que permite usar cualquier función de pérdida diferenciable.

1.4. Gradient Boosting: fundamento matemático

1.4.1. Marco general

Dado un conjunto de datos $\{(x_i, y_i)\}_{i=1}^n$ y una función de pérdida diferenciable $L(y, F(x))$ (por ejemplo, error cuadrático para regresión, log-loss para clasificación), el objetivo es encontrar una función $F(x)$ que minimice:

$$\mathcal{L}(F) = \sum_{i=1}^{n} L(y_i, F(x_i))$$

En lugar de buscar directamente F, se construye de forma iterativa como suma de funciones "base" (generalmente árboles pequeños):

$$F_m(x) = F_{m-1}(x) + \rho_m h_m(x), \quad m = 1, \ldots, M$$

donde $h_m(x)$ es el modelo base (árbol) y ρ_m es el paso de aprendizaje.

1.4.2. Ajuste de los residuos (vista de regresión)

Para el caso particular de pérdida cuadrática $L(y, F) = (y - F)^2/2$, el residuo en la iteración m es:

$$r_{im} = -\left.\frac{\partial L(y_i, F)}{\partial F}\right|_{F=F_{m-1}(x_i)} = y_i - F_{m-1}(x_i)$$

Es decir, el nuevo árbol h_m se entrena para predecir los residuos (errores) del modelo actual. Luego se suma escalado por ρ_m. Esta es la interpretación más sencilla: cada nuevo modelo intenta predecir el error del conjunto anterior.

1.4.3. Funciones de pérdida arbitrarias

Para una pérdida general, el gradient boosting utiliza el **gradiente negativo** de la pérdida con respecto a la predicción actual:

$$g_{im} = -\left.\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right|_{F=F_{m-1}}$$

y entrena el árbol h_m para predecir estos gradientes (o una aproximación). La actualización es:

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$

donde η es la **tasa de aprendizaje (learning rate)**, un factor de contracción que reduce la contribución de cada árbol y permite que el modelo aprenda más lentamente, mejorando la generalización.

1.4.4. Algoritmo genérico de Gradient Boosting

1. Inicializar $F_0(x) = \arg\min_\gamma \sum_{i=1}^n L(y_i, \gamma)$ (una constante).
2. Para $m = 1$ hasta M:
   a) Calcular los pseudo-residuos (gradientes negativos):
   $$g_{im} = -\left.\frac{\partial L(y_i, F)}{\partial F}\right|_{F=F_{m-1}(x_i)}$$
   b) Ajustar un árbol de regresión $h_m(x)$ a los pares (x_i, g_{im}).
   c) Determinar el paso óptimo ρ_m (o usar η fijo).
   d) Actualizar $F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$.
3. Salida $F_M(x)$.

1.4.5. Componentes clave

- **Árboles débiles**: típicamente árboles de poca profundidad (por ejemplo, 3-8 niveles). Cada árbol aporta una pequeña mejora.
- **Learning rate (η)**: controla cuánto contribuye cada árbol. Valores pequeños (0.01-0.1) requieren más árboles, pero generalizan mejor.
- **Número de árboles (M)**: se elige mediante validación cruzada o early stopping.

1.5. Implementaciones modernas: XGBoost, LightGBM y CatBoost

1.5.1. XGBoost (eXtreme Gradient Boosting)

XGBoost, desarrollado por Tianqi Chen, se ha convertido en el algoritmo de referencia en competiciones de Kaggle desde 2014. Sus principales innovaciones son:

Regularización

Añade términos de penalización a la función de pérdida para controlar la complejidad:

$$\mathcal{L} = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

donde $\Omega(f) = \gamma T + \frac{1}{2}\lambda\|w\|^2 + \alpha|w|$, con T número de hojas, w pesos de las hojas, γ, λ, α hiperparámetros. Esto reduce el sobreajuste y mejora la generalización.

Manejo de valores faltantes

XGBoost aprende automáticamente la dirección óptima para tratar los nulos (si van a la izquierda o derecha) durante el entrenamiento, sin necesidad de imputación previa.

Early stopping

Detiene el entrenamiento si el rendimiento en validación no mejora después de un número de rondas, ahorrando tiempo y evitando sobreajuste.

Importancia de características

Proporciona varias métricas: *weight* (número de veces que una variable se usa para dividir), *gain* (ganancia promedio de la variable en las divisiones) y *cover* (cobertura promedio). La más usada es *gain*.

Optimizaciones computacionales

- Estructuras de datos en bloques de caché para acelerar el cálculo de splits.
  item Paralelización en la construcción de árboles.
- Soporte para GPU.

1.5.2. LightGBM (Light Gradient Boosting Machine)

Desarrollado por Microsoft, LightGBM introduce dos técnicas para acelerar el entrenamiento sin sacrificar precisión:

GOSS (Gradient-based One-Side Sampling)

En lugar de usar todas las observaciones para calcular la ganancia de una división, GOSS retiene las observaciones con gradientes grandes (más error) y muestrea aleatoriamente las de gradiente pequeño. Esto reduce el número de observaciones a considerar, acelerando el proceso.

EFB (Exclusive Feature Bundling)

Muchas características son mutuamente excluyentes (nunca toman valores distintos de cero simultáneamente). EFB las agrupa en una sola característica, reduciendo la dimensionalidad y acelerando el entrenamiento.

Soporte nativo para variables categóricas

LightGBM puede manejar directamente variables categóricas sin necesidad de one-hot encoding, mediante histogramas especiales. Esto evita la explosión dimensional.

Crecimiento por hoja (leaf-wise)

A diferencia de XGBoost que crece nivel por nivel (level-wise), LightGBM crece por hoja: en cada iteración expande la hoja que más reduce la pérdida. Esto puede generar árboles más profundos y precisos, pero también puede causar sobreajuste si no se limita (parámetro `num_leaves`).

1.5.3. CatBoost (Categorical Boosting)

Desarrollado por Yandex, CatBoost se especializa en datos con muchas variables categóricas. Sus características principales:

- **Target encoding** con ordenamiento aleatorio para evitar leakage.
- **Árboles simétricos** que aceleran la predicción.
- Manejo robusto de categóricas sin preprocesamiento.

Aunque no se profundizará, es importante conocer su existencia.

1.6. Comparación con Random Forest

Característica	Random Forest	Gradient Boosting (XGBoost)
Construcción	Paralela (bagging)	Secuencial (boosting)
Objetivo	Reducir varianza	Reducir sesgo
Manejo de overfitting	Robusto por naturaleza	Requiere regularización y early stop
Velocidad de entrenamiento	Rápido (paralelo)	Puede ser más lento (secuencial)
Número de árboles	Suele necesitar menos	Puede necesitar miles
Interpretabilidad	Importancia de características	Importancia de características
Datos ruidosos	Robusto	Sensible (puede sobreajustar)
Rendimiento típico	Bueno, pero boosting suele ganar	Excelente, especialmente con ajuste

Cuadro 1.1: Comparación entre Random Forest y Gradient Boosting.

En general, Gradient Boosting suele superar a Random Forest en problemas donde se necesita máxima precisión, pero requiere más cuidado en el ajuste de hiperparámetros.

1.7. Hiperparámetros clave en XGBoost y LightGBM

1.7.1. Hiperparámetros comunes

- **learning_rate** (η): tasa de aprendizaje. Valores típicos: 0.01-0.3. Menor learning rate requiere más árboles.
- **n_estimators** (M): número de árboles. Se ajusta con early stopping.
- **max_depth**: profundidad máxima de cada árbol. Controla la complejidad. Valores típicos: 3-10.
- **subsample**: fracción de observaciones usadas para cada árbol (similar a bagging). Reduce sobreajuste.
- **colsample_bytree**: fracción de características usadas en cada árbol.
- **min_child_weight** (XGBoost) / **min_sum_hessian** (LightGBM): peso mínimo en una hoja para seguir dividiendo. Controla overfitting.

1.7.2. Hiperparámetros específicos de XGBoost

- **lambda** (λ): regularización L2 sobre los pesos de las hojas.
- **alpha** (α): regularización L1.
- **gamma** (γ): reducción mínima de pérdida requerida para hacer una división.

1.7.3. Hiperparámetros específicos de LightGBM

- **num_leaves**: número máximo de hojas por árbol. Relacionado con max_depth, pero permite árboles asimétricos.
- **min_data_in_leaf**: mínimo de observaciones por hoja.
- **feature_fraction**: similar a colsample_bytree.
- **bagging_fraction**: similar a subsample.
- **lambda_l1**, **lambda_l2**: regularización.

1.8. Métricas de evaluación en boosting

1.8.1. Clasificación

- **Log-loss** (pérdida logarítmica): para problemas binarios o multiclase. Mide la calidad de las probabilidades predichas. Se define como:

$$\text{LogLoss} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{p}_i) + (1 - y_i)\log(1 - \hat{p}_i)\right]$$

  Es la función de pérdida por defecto en XGBoost para clasificación. Penaliza más las predicciones seguras pero erróneas.

- **AUC-ROC**: mide la capacidad discriminativa independientemente del umbral. Muy útil en problemas desbalanceados.
- **Precisión, Recall, F1**: según el costo de falsos positivos y falsos negativos.

1.8.2. Regresión

- **RMSE**: sensible a outliers.
- **MAE**: robusto a outliers.
- **RMSLE** (Root Mean Squared Logarithmic Error): útil cuando la variable objetivo tiene crecimiento exponencial (ej. precios, ventas). Se define como:

$$\text{RMSLE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\log(y_i + 1) - \log(\hat{y}_i + 1))^2}$$

  Penaliza más la infraestimación que la sobreestimación.

1.8.3. Ejemplo comparativo de métricas

Supóngase un problema de predicción de ventas (regresión) con valores reales: [100, 200, 300]. Dos modelos predicen:

- Modelo A: [110, 190, 310]
- Modelo B: [90, 210, 290]

$\text{RMSE}_A = \sqrt{(10^2 + 10^2 + 10^2)/3} \approx 10$, $\text{RMSE}_B = \sqrt{(10^2 + 10^2 + 10^2)/3} = 10$. Ambos igual. $\text{MAE}_A = (10 + 10 + 10)/3 = 10, MAE_B = 10.Igual.RMSLE_A : calcularlog(y + 1) : log(101) = 4.615, log(201) = 5.303, log(301) = 5.707; log(p + 1) : log(111) = 4.710, log(191) = 5.252, log(311) = 5.740; diferencias : 0.095, -0.051, 0.033; cuadrados : 0.0090, 0.0026, 0.0011; promedio 0.00423; raíz : 0.065.RMSLE_B similar.Enestecasonohaydiferencia.$

Pero si un modelo subestima sistemáticamente (ej. predice 90, 180, 270), el RMSLE penalizará más porque el logaritmo es más sensible en valores pequeños. Esto es útil en problemas de demanda donde subestimar puede llevar a rotura de stock.

1.9. Aplicaciones prácticas

1.9.1. Predicción de clics en publicidad online (CTR)

En publicidad digital, se dispone de enormes volúmenes de datos (miles de millones de impresiones) con características como usuario, página web, dispositivo, hora, etc. El objetivo es predecir la probabilidad de que un usuario haga clic en un anuncio. XGBoost y LightGBM son los algoritmos más utilizados debido a:

- Escalabilidad a grandes datos.
- Manejo de variables categóricas de alta cardinalidad (LightGBM con EFB).
- Capacidad de capturar interacciones complejas.

La métrica principal suele ser log-loss o AUC.

1.9.2. Competencias Kaggle

La mayoría de las soluciones ganadoras en competiciones de datos tabulares utilizan Gradient Boosting, a menudo en combinación con otros modelos (ensambles). Ejemplos: House Prices, Otto Group Product Classification, Porto Seguro's Safe Driver Prediction.

1.9.3. Modelado de riesgo crediticio

Bancos y financieras utilizan boosting para predecir la probabilidad de incumplimiento de pago. La interpretabilidad no es tan crucial como la precisión, y XGBoost proporciona además importancia de características para entender los factores de riesgo. La métrica clave suele ser AUC o Gini.

1.10. Caso integrado: Predicción de abandono de clientes (churn)

1.10.1. Problema

Una empresa de telecomunicaciones desea predecir qué clientes tienen alta probabilidad de abandonar el servicio en los próximos meses. Dispone de datos demográficos, de uso (minutos, datos consumidos), de servicio al cliente (número de reclamaciones) y económicos (factura media).

1.10.2. Datos

Se tienen 100,000 clientes, con 50 características. La variable objetivo es binaria (1: abandona, 0: no abandona). La clase positiva representa el 15 %.

1.10.3. Preprocesamiento

- Se identifican valores faltantes (algunas variables de uso pueden ser nulas si el cliente no usó ciertos servicios). XGBoost maneja nulos internamente, por lo que no es necesario imputar.
- Variables categóricas: en LightGBM se pueden pasar directamente como categóricas; en XGBoost se codifican con one-hot o label encoding.
- Se divide en entrenamiento (70 %), validación (15 %) y prueba (15 %).

1.10.4. Modelado y ajuste de hiperparámetros

Se prueban XGBoost y LightGBM. Para ambos, se realiza una búsqueda aleatoria o por rejilla sobre:

- learning_rate: [0.01, 0.05, 0.1]
- max_depth: [3, 5, 7]
- subsample: [0.8, 0.9, 1.0]
- colsample_bytree: [0.8, 0.9, 1.0]
- lambda, alpha: [0, 0.1, 1]
- n_estimators: se usa early stopping con 50 rondas de paciencia.

La métrica de validación es AUC (por el desbalance).

1.10.5. Resultados

Tras la optimización, se obtienen los siguientes rendimientos en el conjunto de prueba:

LightGBM supera ligeramente a XGBoost en este problema, posiblemente por su mejor manejo de las variables categóricas y la velocidad que permitió explorar más combinaciones de hiperparámetros.

Modelo	AUC	Log-loss	F1 (clase 1)
XGBoost	0.832	0.352	0.612
LightGBM	0.845	0.341	0.631
Random Forest	0.801	0.382	0.582
Regresión Logística	0.765	0.412	0.521

Cuadro 1.2: Comparación de modelos en datos de prueba.

1.10.6. Importancia de características

Se analiza la importancia por ganancia de XGBoost. Las variables más relevantes son:

1. Número de reclamaciones en los últimos 6 meses.
2. Factura media.
3. Antigüedad del cliente.
4. Consumo de datos en el último mes.

Esto sugiere acciones de retención: ofrecer descuentos a clientes con muchas reclamaciones, o planes personalizados según consumo.

1.10.7. Conclusión del caso

El modelo Gradient Boosting permite identificar con buena precisión los clientes en riesgo, y además proporciona información interpretable para diseñar estrategias de retención.

1.11. Implementación en Python

1.11.1. XGBoost

```
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size
   =0.2, random_state=42)

dtrain = xgb.DMatrix(X_train, label=y_train)
dval = xgb.DMatrix(X_val, label=y_val)

params = {
    'objective': 'binary:logistic',
    'eval_metric': 'logloss',
    'max_depth': 5,
    'learning_rate': 0.05,
    'subsample': 0.8,
    'colsample_bytree': 0.8,
    'lambda': 1.0,
```

```
    'alpha': 0.1
}

model = xgb.train(
    params,
    dtrain,
    num_boost_round=1000,
    evals=[(dval, 'validation')],
    early_stopping_rounds=50,
    verbose_eval=50
)

y_pred = model.predict(dval)
auc = roc_auc_score(y_val, y_pred)
print(f"AUC: {auc:.4f}")
```

1.11.2. LightGBM

```
import lightgbm as lgb
from sklearn.metrics import roc_auc_score

train_data = lgb.Dataset(X_train, label=y_train)
val_data = lgb.Dataset(X_val, label=y_val, reference=train_data)

params = {
    'objective': 'binary',
    'metric': 'auc',
    'boosting_type': 'gbdt',
    'num_leaves': 31,
    'learning_rate': 0.05,
    'feature_fraction': 0.8,
    'bagging_fraction': 0.8,
    'bagging_freq': 5,
    'lambda_l1': 0.1,
    'lambda_l2': 1.0,
    'verbose': -1
}

model = lgb.train(
    params,
    train_data,
    valid_sets=[val_data],
    num_boost_round=1000,
    callbacks=[lgb.early_stopping(50), lgb.log_evaluation(50)]
)

y_pred = model.predict(X_val)
auc = roc_auc_score(y_val, y_pred)
print(f"AUC: {auc:.4f}")
```

1.12. Conexión con el resto del curso

Gradient Boosting es el punto culminante de los modelos basados en árboles. Con él se cierra el ciclo de modelos supervisados clásicos. En las siguientes sesiones se abordarán:

- **Redes neuronales**: que pueden verse como una forma de boosting continuo (capas apiladas) pero con optimización conjunta.
- **Aprendizaje no supervisado**: donde los árboles también tienen aplicaciones (ej. Isolation Forest para detección de anomalías).
- **Interpretabilidad avanzada**: técnicas como SHAP, que se construyeron inicialmente para explicar XGBoost.

1.13. Resumen

- Gradient Boosting construye modelos secuencialmente, cada uno corrige los errores del anterior mediante el gradiente de la pérdida.
- XGBoost añade regularización, manejo de nulos y optimizaciones computacionales, convirtiéndolo en el algoritmo más popular.
- LightGBM introduce GOSS y EFB para acelerar el entrenamiento sin perder precisión, y maneja categóricas nativamente.
- La elección entre ellos depende del tamaño de datos, la presencia de categóricas y la necesidad de velocidad.
- Los hiperparámetros deben ajustarse cuidadosamente, usando early stopping para evitar sobreajuste.
- Las métricas adecuadas dependen del problema: log-loss y AUC para clasificación; RMSE, MAE o RMSLE para regresión.
- En aplicaciones prácticas, estos modelos son la herramienta por defecto para datos tabulares, y su dominio es esencial para cualquier científico de datos.